# Inside the app

Every screen, every switch — and why it was built that way.

Document 3 of 4 · Version 1.0 · Contact: keshavgithub01@gmail.com

This document walks through LibRate screen by screen. Each section ends with a **Why** note: the design decision behind it. Several of these choices look odd for a normal messenger; they make sense for a tool whose users may be in a crowd, offline, and under observation.

## 1. First run

Three short screens: *what this is*, *your name*, *three permissions*. A key pair is generated silently before the first screen appears; the name step shows the generated name with a pen icon to change it. The permissions step asks for Nearby devices, Notifications and Background activity in one go ("Allow all") or can be skipped ("Not now").

**Why a generated name?** People under pressure type their real name out of habit. Offering a ready-made one (*Brave Otter*) makes the anonymous choice the default. **Why ask permissions after a screen of explanation?** Android's dialogs are terse; a user who understands why Bluetooth is needed is far more likely to grant it, and far less likely to leave the app broken.

## 2. Nearby

One public room for everyone in Bluetooth range plus whatever relays bridge in. Each message shows the sender's name and short ID, how it arrived ( `BT` / `relay` ) and the hop count. The **Bluetooth card** at the top reports: off → turn on; permission missing → allow; searching; or *N links · M people seen*. Nearby never sends notifications. Triple-tapping the title offers a wipe.

**Why no notifications for Nearby?** In a crowd it would buzz constantly and train people to ignore all alerts. Private chats and joined rooms notify; Nearby is for glancing. **Why show hop count and transport?** It is live feedback that the mesh is working, and it tells you how far your words are reaching. **Why the short ID next to names?** Anyone can pick any nickname; the ID (derived from the key) is what cannot be faked. Duplicate nicknames are flagged automatically.

## 3. Rooms

- **Open rooms** — by name. Signed, public.
- **Password rooms** — name + password, encrypted. The password never leaves the phone.
- **Location rooms** — *Find my area* takes one coarse GPS fix and derives rooms for city, neighbourhood and block. Nothing is uploaded; the fix is not stored.
- **Groups** — rooms with a creator and admins; members can be removed; the creator can dismantle. Shared by QR invite.

**Why location rooms from a single fix?** People in the same place need a room they can all guess without coordination. One coarse fix, used once and discarded, gives that without tracking anyone. **Why can admins remove members but there is no server?** The creator signs a charter listing admins; removals are signed too. Each phone checks the signatures and drops frames from removed members. Moderation without a moderator server. **Why do invites carry relay addresses?** So a group formed offline keeps working when members scatter and go online — they all land on the same relays.

## 4. Chats

Private, end-to-end encrypted. Start one by tapping a name in a room, from *People*, or by scanning a QR. Messages from strangers go under **Message requests** (one row with a count; tap to review, accept or decline-and-block). Ticks: pending → sent → delivered → seen; your bubbles change tint once read. Long-press a chat to favourite, mute, block or delete.

**Why message requests?** In a public mesh anyone can message anyone. Requests stop spam and harassment from reaching the main list while still letting a genuine stranger reach you. **Why are read receipts encrypted?** Otherwise a relay could see *that* you read a message, even without seeing its content.

## 5. People

Everyone seen in the last ten minutes: name, ID, avatar, Bluetooth/relay badge, hop distance. Tap for a profile with bio and QR; favourite, mute, block, or message.

**Why ten minutes?** Long enough to find someone who just spoke; short enough that the list reflects who is actually around now.

## 6. Settings

| | |
|---|---|
| Identity | Name, avatar, bio, your QR. **New identity**: a fresh key that keeps your name and can hand your contacts over with a signed note from the old key. |
| Relays | Built-in relays shown as `Relay 1 ●●●●●●`; user-added relays show their host. Add by address or QR; share yours as QR; import a list by QR. |
| Notifications | Master switch, per-chat mutes, and a health check for background permission with a snooze (3 days / 1 week / never). |
| Messages | Keep forever (default) · auto-delete after N days · memory only. |
| Appearance | Auto / Light / Dark. |
| Privacy | Block screenshots (default on) · Cover traffic · Battery saver. |
| Danger zone | Panic wipe. |
| About | What LibRate is, the SHA-256 of this exact build, and where to get fresh links. |

### Why are the built-in relay addresses hidden?

Three reasons. **(1) Blocking.** A relay address that appears in every screenshot and every forwarded message is trivial to add to a blocklist. Keeping addresses out of the UI means they spread only inside QR codes and invites, between people who already trust each other. **(2) Safety of relay operators.** Whoever runs a relay is the only identifiable party in the system. Not displaying the hostname gives them a layer of distance. **(3) Honesty about what the user controls.** You cannot edit or remove built-in relays anyway — they are a floor, not a choice — so showing the address invites copying it into places it should not go. Your *own* added relays are shown in full because you chose them.

### Why is "New identity" separate from wipe?

Sometimes you want to break the link to your past key (it may have been seen somewhere) without losing your friends. Rotation signs a hand-off note with the old key so contacts' phones re-point to the new one automatically. Groups you created are dismantled, because the creator key is gone — that is the one thing a rotation cannot carry over, by design.

### Why block screenshots by default?

It stops the app from appearing in the recents switcher and in casual screen grabs — the most common way a chat leaks when a phone is glanced at or briefly taken. It can be turned off in Privacy for people who want to share screenshots deliberately.

### Why is "Keep messages" default *forever* rather than ephemeral?

Field testing showed people losing instructions they needed an hour later. Messages are encrypted at rest, and panic wipe is one tap away, so the safer default is to keep. *Memory only* exists for those who need nothing on disk.

### Why cover traffic and battery saver?

Cover traffic sends random dummy frames so an observer cannot tell when you actually speak. It costs battery, so it is off by default. Battery saver scans less often (2 s every 22 s) for long days.

### Why show the SHA-256 of the installed build?

APKs are shared hand to hand. Two people can compare the hash on their screens and know they are running the identical file — no website or app store needed to establish trust.

### Why a Telegram handle and an e-mail in the app?

Download links get blocked; people need a way to ask for a new one. These are the only identifying strings in the app, and they are there deliberately. Always verify what you are sent against the hash and certificate.

## 7. Slash commands

`/nick Name` change name · `/join room` · `/who` list people · `/msg Name` open a private chat · `/block Name` · `/clear` this view · `/wipe` panic wipe.

**Why keep commands at all?** They are faster than menus with one thumb while walking, and they are the same muscle memory as IRC and BitChat.

## 8. Decisions that shape everything

| | |
|---|---|
| **Text only** | Small frames survive bad radio, fit many hops into little air time, and keep store-and-forward cheap. Media would clog the mesh for everyone. |
| **No accounts, no servers of our own** | Nothing to subpoena, seize, or switch off. The relays are public, replaceable and run by anyone. |
| **7 hops** | The same trade-off BitChat chose: beyond ~10 hops radio collisions cost more than reach gains. Store-and-forward and a single online phone cover the distance instead. |
| **Nostr as the relay protocol** | Open, tiny, already deployed worldwide, and uses the same keys we already have. No custom server to write or trust. |
| **Phone-to-phone distribution** | The app's own channel can be blocked; a crowd cannot. The APK, its hash and this documentation are meant to travel together. |
| **No cloud backup** | A backup is a copy of your private key somewhere you do not control. Losing the phone means losing the identity — that is the price of there being exactly one copy. |
| **Obfuscated release build** | Makes a tampered fork harder to produce quickly. The source remains open; obfuscation protects the binary, not the code. |
| **Open source, protected name** | Anyone may read, build and fork. Forks must use a different name and logo so people can tell a genuine build from a modified one. |